HERRAMIENTA DE AUTOMATIZACIÓN · PROGRAMACIÓN 1 · UCSE

UCSE CLI

Automatización del armado de proyectos .NET para la cursada

Una herramienta de línea de comandos que crea la estructura de soluciones, proyectos y capas que usa la materia, configura las referencias y deja código de ejemplo listo para adaptar — para que el tiempo se invierta en resolver el problema, no en preparar el andamiaje.

C# · .NET | Arquitectura por capas | NUnit | Web API REST | Persistencia JSON | HTML + CSS + JS | Git

Versión 2.0.0
Distribuida como .NET Tool en NuGet

Autor: Bautista
github.com/BauDev0/UcseCLI

1 Qué es y qué problema resuelve

UCSE CLI es una herramienta de línea de comandos que se instala como *.NET Tool* y se invoca con el comando `ucse`. Su objetivo es eliminar el trabajo mecánico que aparece al empezar cada práctica, parcial o trabajo integrador de Programación 1.

Arrancar una solución de la materia implica, siempre, la misma secuencia manual:

```
# lo que hay que escribir a mano en cada práctica nueva
dotnet new sln -n SistemaVentas
dotnet new classlib -n SistemaVentas.LogicaServicio
dotnet new nunit    -n SistemaVentas.LogicaTest
dotnet new console  -n SistemaVentas.Consola
dotnet sln add SistemaVentas.LogicaServicio/SistemaVentas.LogicaServicio.csproj
dotnet sln add SistemaVentas.LogicaTest/SistemaVentas.LogicaTest.csproj
dotnet sln add SistemaVentas.Consola/SistemaVentas.Consola.csproj
dotnet add SistemaVentas.LogicaTest/... reference SistemaVentas.LogicaServicio/...
dotnet add SistemaVentas.Consola/...    reference SistemaVentas.LogicaServicio/...
dotnet add SistemaVentas.LogicaServicio/... package Newtonsoft.Json
# + crear Entidades/, Servicios/, Datos/, el .gitignore, el README...
```

Son diez comandos largos, propensos a errores de tipeo en las rutas, que hay que repetir en cada entrega y que no aportan nada al aprendizaje. Con la herramienta, todo eso es una línea:

```
ucse new SistemaVentas
```

El criterio de diseño: automatizar, no resolver

La herramienta **no programa el trabajo práctico**. Esa distinción es deliberada y marca el límite de lo que hace:

Sí hace

- Crear soluciones, proyectos y carpetas de capas.
- Configurar referencias entre proyectos y paquetes NuGet.
- Generar archivos `.cs` con el namespace correcto.
- Dejar código de ejemplo comentado, con marcas `TODO`.
- Mostrar fragmentos de referencia de cada unidad.

No hace

- Diseñar el modelo del enunciado.
- Implementar las reglas de negocio.
- Escribir los tests de esas reglas.
- Tomar decisiones de diseño orientado a objetos.
- Resolver parciales ni entregas.

> **Por diseño:** el código generado es una *plantilla de referencia* sobre una entidad de ejemplo (`Producto` por defecto). La herramienta lo avisa al terminar: «El modelo y las reglas de negocio de tu enunciado los diseñás vos: borrá o reemplazá lo que no uses». Sirve como punto de partida y como muestra de las convenciones, no como solución.

2 Instalación y primeros pasos

Requisitos

REQUISITO	DETALLE
.NET SDK	Versión 8 o superior. La materia trabaja con .NET 8; la herramienta también funciona con SDK 9 y 10 y se adapta a lo que encuentre instalado.
Git	Opcional. Si está disponible, la herramienta inicializa el repositorio de la solución nueva.

Instalar, actualizar y verificar

```
# instalar
dotnet tool install --global UcseCLI

# actualizar a la última versión
dotnet tool update --global UcseCLI

# verificar: muestra el SDK detectado y la guía de comandos
ucse info
```

`ucse info` cumple dos funciones: diagnostica el entorno (si no encuentra el SDK, lo dice y enlaza la descarga) y hace de índice de comandos. El mismo resumen aparece al escribir `ucse` sin argumentos.

Un recorrido de dos minutos

```
ucse new SistemaVentas          # solución completa por capas
cd SistemaVentas
dotnet test                       # los tests de ejemplo ya pasan

ucse add-entidad SistemaVentas.LogicaServicio/Entidades Cliente
ucse add-servicio SistemaVentas.LogicaServicio/Servicios Cliente
ucse add-test SistemaVentas.LogicaTest ClienteService

ucse add-api                      # suma la Web API a la solución
dotnet run --project SistemaVentas.Api
```

Ayuda en cada comando: `ucse <comando> --help` lista los argumentos y opciones disponibles, con sus valores por defecto.

3 La estructura que genera

La herramienta materializa la arquitectura por capas de los apuntes: la presentación no tiene reglas de negocio, la lógica no sabe de consola ni de HTTP, y el acceso a los datos está encapsulado en una única clase. Esa separación es lo que permite que la misma lógica se exponga después por consola o por una Web API sin reescribirla.

PROYECTO	CAPA Y RESPONSABILIDAD	UNIDAD
`<N>.Consola`	**Presentación.** Muestra información y toma entradas del usuario.	1 y 2
`<N>.Api`	**Presentación HTTP.** Controllers y DTOs; traduce pedidos HTTP a llamadas al servicio.	6
`<N>.LogicaServicio`	**Lógica y datos.** Entidades, servicios con las reglas y repositorios con la persistencia.	2 y 5
`<N>.LogicaTest`	**Tests.** Proyecto NUnit que referencia la lógica (nunca al revés).	4

Solución por capas — `ucse new SistemaVentas`

```
SistemaVentas
├── SistemaVentas.sln
├── .gitignore                      ← .gitignore de .NET (Unidad 3)
├── README.md                       ← estructura y cómo ejecutar
│
├── SistemaVentas.Consola           → presentación
│   └── Program.cs                  menú de ejemplo
│
├── SistemaVentas.LogicaServicio    → lógica de negocio
│   ├── Entidades/Producto.cs
│   ├── Servicios/ProductoService.cs
│   └── Datos/ProductoRepositorio.cs persistencia en JSON
│
└── SistemaVentas.LogicaTest        → tests NUnit
    └── ProductoServiceTest.cs      5 tests de ejemplo, en verde
```

Solución con Web API — `ucse new-webapi SistemaVentas`

```
SistemaVentas.Api
├── Controllers/ProductoController.cs   GET, POST, PUT, DELETE
├── DTOs/ProductoRequest.cs             entrada + Data Annotations
├── DTOs/ProductoResponse.cs            salida
├── DTOs/ProductoMapper.cs              mapeo DTO ↔ entidad
├── SistemaVentas.Api.http              peticiones listas para probar
└── Program.cs                          controllers + CORS + DI
```

Los DTOs quedan en el proyecto de la API y el mapeo se hace en el controller, de modo que el servicio sigue trabajando solo con la entidad: así se evita la referencia circular entre la lógica y la API, que es el punto que el apunte de la Unidad 6 desarrolla en detalle.

4 Cómo funciona

La herramienta no reemplaza al SDK de .NET: lo orquesta. Invoca los mismos comandos `dotnet` que escribiría el alumno, en el orden correcto, y después completa lo que las plantillas oficiales no cubren: las carpetas de capas, los paquetes, los archivos de ejemplo y los archivos del repositorio.

El flujo de `ucse new`

1. **Validación previa**
   Se verifica el nombre (identificador válido, sin palabras reservadas de C#), el *target framework* y que la carpeta no exista. Si algo no cierra, no se crea nada.

2. **Creación de los proyectos**
   Se ejecutan `dotnet new sln`, `classlib`, `nunit` y `console` (o `webapi`), y se borran los archivos de muestra de las plantillas (`Class1.cs`, `UnitTest1.cs`, `WeatherForecast`).

3. **Solución y referencias**
   Se agregan los proyectos a la solución y se configuran las referencias en el único sentido válido: tests → lógica, presentación → lógica, API → lógica.

4. **Paquetes y capas**
   Se agrega `Newtonsoft.Json 13.0.3` al proyecto de lógica y se crean las carpetas `Entidades/`, `Servicios/` y `Datos/`.

5. **Código de ejemplo**
   Se escriben la entidad, el repositorio, el servicio, los tests y el `Program.cs`, cada uno con su namespace y sus directivas `using` ya resueltas.

6. **Repositorio**
   Se generan el `.gitignore` y el `README.md` con la estructura y los comandos de ejecución, y se inicializa Git.

7. **Informe final**
   Se imprime el árbol de lo creado, los próximos comandos a ejecutar y el recordatorio de que los ejemplos son para adaptar.

Decisiones que toma por su cuenta

SITUACIÓN	QUÉ HACE LA HERRAMIENTA
Deducir el namespace	Los comandos `add-*` buscan el `.csproj` más cercano hacia arriba y componen el namespace con las carpetas intermedias: un archivo en `Ventas.LogicaServicio/Entidades` queda en `Ventas.LogicaServicio.Entidades`. Se puede forzar con `--namespace`.
Encontrar la lógica	`add-api` y `add-controller` leen las referencias declaradas en el `.csproj` para saber contra qué proyecto de lógica generar los `using`. Si no pueden determinarlo, lo piden con `--logica`.
Swagger u OpenAPI	Según el paquete que traiga la plantilla del SDK instalado, escribe el `Program.cs` con Swashbuckle (Swagger UI en `/swagger`) o con `AddOpenApi()`, e indica cómo sumar la interfaz de Swagger si hace falta.
Archivo de solución	Detecta si el SDK generó `.sln` o el formato nuevo `.slnx`, y lo informa en el árbol y en el README.
Versión de .NET	Por defecto usa la del SDK instalado. Con `--framework net8.0` se fija la versión que pide la cátedra.

> **Un detalle que la herramienta resuelve sola:** desde .NET 8 la plantilla `dotnet new webapi` genera *Minimal APIs*, sin carpeta `Controllers`. La materia trabaja con controllers, así que la herramienta crea el proyecto con `--use-controllers` y escribe ella misma el `Program.cs` con `AddControllers()`, CORS habilitado para el frontend y el servicio registrado en el contenedor de dependencias.

Resguardos

- **No sobreescribe trabajo.** Los comandos `add-*`, el `.gitignore` y el `README.md` respetan los archivos existentes: avisan y los dejan intactos.
- **No escribe fuera de la carpeta actual.** Toda ruta destino se resuelve y se verifica contra el directorio de trabajo, así un `../..` no puede tocar nada de afuera.
- **Validación antes de escribir.** Si uno de varios nombres es inválido, no se crea ninguno.
- **Errores entendibles.** Si falta el SDK, si un comando de `dotnet` falla o si no se puede escribir en el disco, se explica el problema y el paso siguiente, sin volcar un *stack trace*.

5 Referencia de comandos

Armar y ampliar soluciones

COMANDO	QUÉ GENERA
`ucse new <Nombre>`	Solución por capas: consola + lógica + tests NUnit.
`ucse new-webapi <Nombre>`	Solución con Web API: API (controllers y DTOs) + lógica + tests.
`ucse add-api`	Suma un proyecto Web API a una solución que ya existe.
`ucse new-frontend [carpeta]`	Frontend en HTML, CSS y JavaScript que consume la API.

Opciones

OPCIÓN	EFECTO
`-e`, `--entidad <N>`	Entidad del código de ejemplo (por defecto `Producto`).
`-f`, `--framework <tfm>`	*Target framework*, por ejemplo `net8.0`.
`--con-api` / `--con-consola`	Suma el proyecto de API (en `new`) o el de consola (en `new-webapi`).
`--sin-consola`	Solo lógica y tests.
`--sin-ejemplos`	Genera la estructura vacía, sin código de ejemplo.
`--sin-git`	No inicializa el repositorio.
`--api <url>`	URL del endpoint que consume el frontend.
`--logica <Proyecto>`	Proyecto de lógica, cuando no se puede detectar solo.

Agregar código a un proyecto

Todos deducen el namespace del `.csproj` y de la carpeta destino, y aceptan `-n`, `--namespace` para forzarlo.

COMANDO	GENERA
`ucse add-clase <ruta> <Nombres...>`	Clases vacías. Alias histórico: `add-files`.
`ucse add-entidad <ruta> <Nombres...>`	Entidades del dominio (POCO con `Id`).
`ucse add-interfaz <ruta> <Nombres...>`	Interfaces, agregando el prefijo `I`.
`ucse add-abstracta <ruta> <Nombres...>`	Clases abstractas con un miembro abstracto y uno concreto.
`ucse add-enum <ruta> <Nombres...>`	Enumeradores.
`ucse add-servicio <ruta> <Entidad>`	Servicio con alta, baja, modificación, búsqueda y validaciones.
`ucse add-repositorio <ruta> <Entidad>`	Repositorio con persistencia en JSON (y el paquete, si falta).
`ucse add-test <ruta> <Clase>`	`TestFixture` de NUnit con patrón Arrange-Act-Assert.
`ucse add-controller <proyecto-api> <Entidad>`	Controller REST más `Request`, `Response` y `Mapper`.

Atajo útil: si el nombre que recibe `add-test` termina en `Service`, la herramienta genera la batería completa de tests de ejemplo (alta con Id asignado, validación que lanza excepción, búsqueda inexistente que devuelve `null`, baja y verificación de que los datos realmente persistieron), cada uno sobre un archivo JSON temporal para que queden aislados entre sí.

Referencia y utilidades

COMANDO	PARA QUÉ SIRVE
`ucse ejemplos [tema]`	Muestra fragmentos de código de la materia; con `--guardar <ruta>` los escribe en un archivo.
`ucse gitignore`	Crea un `.gitignore` de .NET en la carpeta actual.
`ucse info`	Estado del entorno y guía de comandos.
`ucse --version`	Versión instalada de la herramienta.

6 El catálogo de ejemplos reutilizables

Además de lo que escribe en los proyectos, la herramienta incluye un catálogo de fragmentos de referencia, pensados para consultar sin salir de la terminal y copiar o guardar a un archivo. Cubren las unidades 2 a 7:

TEMA	CONTENIDO	UNIDAD
`herencia`	Clase abstracta, `override` y polimorfismo sobre una lista del tipo base.	2
`interfaz`	Contrato, dos implementaciones y uso por constructor sin depender de la concreta.	2
`excepciones`	`ArgumentException` frente a `InvalidOperationException`, y `try/catch/finally`.	2
`linq`	`Where`, `Select`, `OrderBy`, `FirstOrDefault`, agregados y `GroupBy`.	2
`nunit`	Arrange-Act-Assert, `Assert.Throws`, `TestCase` y comparación de decimales con `Within`.	4
`json`	Repositorio con `JsonConvert` y `File`, y el patrón leer → modificar → guardar.	5
`controller`	Controller REST con DTOs, códigos de estado y filtrado por *query string*.	6
`fetch`	`fetch` con GET, POST y DELETE, y armado del DOM sin inyectar HTML.	7
`git`	Ramas, commits, *pull requests* y sincronización con el equipo.	3

```
ucse ejemplos                              # lista los temas disponibles
ucse ejemplos linq                         # lo muestra en la terminal
ucse ejemplos json --guardar Notas/json.cs  # lo guarda en un archivo
```

El objetivo de este catálogo es doble: evitar la búsqueda del apunte a mitad de un ejercicio y fijar una forma correcta de escribir cada cosa, con los comentarios que explican *por qué* se hace así.

Alineación con el material de la materia

Todo lo que genera sigue las convenciones del repositorio oficial de la cátedra (*UCSE-PROG1/programacion-1*): nombres en PascalCase y camelCase según corresponda, interfaces con prefijo `I`, un archivo por clase, separación en capas, el patrón «leer todo → modificar en memoria → guardar todo» para la persistencia, tests con Arrange-Act-Assert y nombres `Metodo_Escenario_ResultadoEsperado`, y DTOs del lado de la Web API con el mapeo hecho en el controller.

7 Casos de uso

Caso 1 — Primer trabajo con objetos y consola

Unidades 1 y 2. Hay que entregar una aplicación de consola con clases, propiedades, colecciones y LINQ, separando la presentación de la lógica.

```
ucse new Biblioteca --entidad Libro
cd Biblioteca
ucse add-entidad Biblioteca.LogicaServicio/Entidades Autor Prestamo Usuario
ucse add-enum Biblioteca.LogicaServicio/Entidades EstadoPrestamo
dotnet run --project Biblioteca.Consola
```

Queda una solución que compila y corre desde el primer minuto, con el menú de consola llamando al servicio y las entidades listas para completar con las propiedades del enunciado.

Caso 2 — Práctica con persistencia y tests

Unidades 4 y 5. La práctica pide lógica de negocio con reglas, persistencia en archivos JSON y tests unitarios que cubran los casos válidos y los que deben rechazarse.

```
ucse new HelpDesk --entidad Ticket --sin-consola
cd HelpDesk
ucse add-enum HelpDesk.LogicaServicio/Entidades Prioridad Estado
ucse add-test HelpDesk.LogicaTest TicketService
dotnet test
```

El repositorio JSON, el servicio y los tests de ejemplo ya implementan el patrón que pide el apunte —incluido el caso de la primera ejecución, cuando el archivo todavía no existe— y los tests corren sobre archivos temporales, así que no se pisan entre sí ni contaminan los datos reales. Sobre esa base se agregan las reglas del enunciado.

Caso 3 — Exponer la lógica como Web API

Unidad 6. La práctica siguiente pide sumar un proyecto Web API a la solución anterior, sin reescribir lo hecho.

```
cd HelpDesk
ucse add-api
dotnet run --project HelpDesk.Api
```

La herramienta detecta la solución y el proyecto de lógica, crea la API con controllers, la referencia correctamente, genera el controller con sus DTOs y su mapper a partir del servicio existente, y deja el `Program.cs` con la inyección de dependencias y CORS resueltos. El archivo `.http` que genera permite probar los cinco endpoints sin configurar nada más.

Caso 4 — Agregar un recurso nuevo a una API existente

Unidad 6. Aparece una segunda entidad que también hay que exponer por HTTP, con su propio controller y sus DTOs.

```
ucse add-repositorio HelpDesk.LogicaServicio/Datos Usuario
ucse add-servicio HelpDesk.LogicaServicio/Servicios Usuario
ucse add-controller HelpDesk.Api Usuario
ucse add-test HelpDesk.LogicaTest UsuarioService
```

Cuatro comandos generan las cuatro capas del recurso, cada archivo en su carpeta y con los `using` correctos entre proyectos. La herramienta recuerda al final el único paso manual que queda: registrar el servicio nuevo en `Program.cs`.

Caso 5 — Frontend que consume la API

Unidad 7. Hay que armar las pantallas en HTML, CSS y JavaScript sin frameworks, consumiendo la API con `fetch`.

```
ucse new-frontend Frontend --entidad Ticket --api http://localhost:5050/api/ticket
```

Genera `index.html` con formulario y tabla, `styles.css` y un `app.js` con GET, POST y DELETE, manejo de errores de red y construcción del DOM con `textContent` en lugar de `innerHTML`. Sirve como esqueleto de las pantallas y como ejemplo de las tres operaciones básicas.

Caso 6 — Trabajo práctico integrador en equipo

Unidades 1 a 7. El TP final pide dos soluciones .NET independientes —una API de gestión y otra de validación, cada una con su lógica y sus tests— más el frontend, todo en un repositorio con carpetas en la raíz.

```
mkdir 2026-prog1-tpfinal-equipo && cd 2026-prog1-tpfinal-equipo
ucse gitignore
ucse new-webapi ApiGestion    --entidad Evento  --sin-git
ucse new-webapi ApiValidacion --entidad Entrada --sin-git
ucse new-frontend Frontend --entidad Evento
```

En cuatro comandos queda armada la estructura que pide el enunciado, con las dos soluciones completas y el frontend. El equipo arranca directamente por el análisis del dominio y las reglas de negocio, que es lo que se evalúa, y no pierde la primera semana resolviendo diferencias de estructura entre integrantes.

Caso 7 — Parcial o práctica a contrarreloj

Cualquier unidad. En una evaluación con tiempo acotado, los minutos del armado inicial son minutos que se le restan al ejercicio.

```
ucse new PreParcial --entidad Vuelo
ucse ejemplos nunit
```

La estructura, las referencias y el proyecto de tests quedan listos de entrada, y el catálogo de ejemplos reemplaza la búsqueda del apunte cuando hay que recordar una firma o un *assert* puntual.

8 Confiabilidad y construcción

Verificación automática

Una herramienta que genera código tiene una exigencia particular: no alcanza con que ella compile, tiene que compilar *lo que produce*. El repositorio incluye una integración continua que, en cada cambio, instala la herramienta empaquetada y la usa como la usaría un alumno:

VERIFICACIÓN	RESULTADO
Compilación de la herramienta	0 advertencias, 0 errores
`ucse new` → `dotnet build` + `dotnet test`	compila; 5 tests en verde
Los nueve comandos `add-*` sobre esa solución	compila; 10 tests en verde
`ucse add-api` y `ucse add-controller`	compila
`ucse new-webapi` con consola	compila; 5 tests en verde
Estructura con `--sin-ejemplos --con-api`	compila
`ucse new-frontend`	genera los archivos esperados

Qué garantiza esto: si la herramienta genera una solución, esa solución compila y sus tests de ejemplo pasan. El alumno nunca arranca desde un proyecto roto.

Tecnologías

COMPONENTE	TECNOLOGÍA
Lenguaje y plataforma	C# sobre .NET 10 (los proyectos generados, .NET 8 o superior)
Análisis de la línea de comandos	System.CommandLine
Salida por terminal	Spectre.Console
En los proyectos generados	NUnit, Newtonsoft.Json, ASP.NET Core
Distribución	.NET Tool publicada en NuGet

Organización del código fuente

```
UcseCLI/UcseCLI
├── Program.cs      punto de entrada
├── Cli/            definición de los comandos y los informativos
├── Core/           validaciones, rutas seguras, dotnet y git, salida
├── Generators/     armado de soluciones, API, frontend y archivos
└── Templates/      plantillas de código y catálogo de ejemplos
```

Las plantillas se escriben como texto literal con marcadores (`__ENTIDAD__`, `__NS__`), así el código de ejemplo se lee en el fuente igual que el archivo que se va a generar: revisarlo o corregirlo no exige descifrar cadenas con saltos y llaves escapadas.

Hoja de ruta

FUNCIONALIDAD	ESTADO
Integración con Web API	Terminado
Interfaces, clases abstractas y enumeradores	Terminado
Arquitectura multicapa	Terminado
Persistencia JSON y tests de ejemplo	Terminado
Frontend HTML, CSS y JavaScript	Terminado
Integración con Git	Terminado
Plantillas personalizadas definidas por el usuario	Pendiente
Generación de clientes `HttpClient`	Pendiente

UCSE CLI 2.0.0 · Proyecto desarrollado para automatizar tareas repetitivas durante la cursada de Programación 1 (UCSE) · Licencia MIT · github.com/BauDev0/UcseCLI